LMA Assignment: RAG with Baseline Retrieval vs. Re-Ranking

Sathvik Reddy | Roll No.: 2025201020

GitHub: `https://github.com/Sathviktr/RAG_LMA`

October 2026

1 Setup and what I built

For this assignment I picked the U.S. Department of Labor's *Employer's Guide to the Family and Medical Leave Act* (76 pages). I chose it because almost every rule in it has some condition or exception, and the same phrases come back with different meanings ("12 months of employment", "12-month period", "12 workweeks"). I thought this would make it easy to find cases where retrieval goes wrong.

I read the PDF with PyMuPDF. The raw text had a lot of noise: the running header and footer on every page, the table of contents, the index, and text from diagrams and comic strips. I removed these using the position and font size of the text, which gave me 474 paragraphs. Then I split the text into 207 chunks of at most 200 tokens (counted with the bi-encoder's own tokenizer, because it cuts off anything after 256 tokens). Each chunk overlaps the previous one by up to 40 tokens and starts with its section heading.

Both pipelines use the same chunks, the same prompt and the same LLM:

- **Pipeline 1 (baseline):** `all-MiniLM-L6-v2` embeddings in a FAISS inner-product index (normalised vectors, so it is cosine similarity). Retrieve the top 10, send the first 3 to the LLM (Answer 1).
- **Pipeline 2 (re-ranking):** take the same top 10, score each (question, chunk) pair with `cross-encoder/ms-marco-MiniLM-L-6-v2`, sort by that score and send the new top 3 to the LLM (Answer 2).

For the LLM I first used Gemini 3.8 Flash, but its free tier only allows 20 requests per day and I ran out while testing, so I switched to Gemini 3.5 Flash-Lite (temperature 0) for everything. The prompt tells the model to use only the given chunks, to cite them like [C140], and to say "The provided context does not contain the answer." if the answer is not there. All answers are cached, so re-running the notebook gives exactly the same answers.

To test the pipelines I wrote 21 questions before running anything, so that I could not just keep the questions where re-ranking looked good. Each question is aimed at one failure type: keyword trap (K), exception located somewhere else (E), negation or constraint (N), or a normal control question (C). For every question I picked an exact sentence from the guide that answers it. The "gold" chunk is the chunk containing that sentence, so the labels are easy to check.

2 Architecture comparison: bi-encoder vs. cross-encoder

A **bi-encoder** encodes the question and the chunk separately. Each text goes through the transformer and the token vectors are averaged into a single 384-dimensional vector. The relevance score is just the cosine similarity between the two vectors. The big advantage is that the chunk vectors do not depend on the question, so I could compute all 207 of them once (about 5 seconds) and store them. For a new question only the question needs to be encoded, and then FAISS compares it with the stored vectors in well under a millisecond.

The problem is that the whole chunk is squashed into one average vector and the question and chunk never interact. Small words like "not" hardly change the average. I tested this: "The employer may count FMLA leave under a no-fault attendance policy" and "The employer may not count FMLA leave . . . " got a cosine similarity of 0.93, even though they say the opposite.

A **cross-encoder** instead takes the question and the chunk together as one input, `[CLS] question [SEP] chunk [SEP]`, and outputs one relevance score (a logit). Because it is one sequence, self-attention lets every question token attend to every chunk token in all 6 layers, so the model can compare the details of the two texts directly. For the question "My wife and I work for the same company. Can each of us take 12 weeks?", it gave the sentence about spouses sharing "a combined total of 12 workweeks" a score of +3.2 and the sentence "Fathers are equally entitled to take up to 12 workweeks" −2.2. The bi-encoder gave them 0.68 and 0.52, which is much closer. The downside is that nothing can be precomputed: every (question, chunk) pair needs a full forward pass for every new question. I also noticed it measures relevance, not truth. "Employers are prohibited from counting . . . " and "Employers may count . . . " both scored around 9, because both are about the question.

	Bi-encoder	**Cross-encoder**
Input	question and chunk separately	question + chunk together
How they interact	only via cosine of final vectors	full self-attention between tokens
Precompute chunks?	yes, once	no
Cost per question	1 encoding + vector search	1 forward pass per candidate
Good at	speed, large collections	details like "not", "same employer"

Table 1: Bi-encoder vs. cross-encoder (both models have 6 layers and 22.7M parameters).

3 Rank shift analysis and Answer 1 vs. Answer 2

	Hit@3	**Recall@10**	**MRR@10**
Pipeline 1 (bi-encoder)	0.571 (12/21)	0.857	0.574
Pipeline 2 (with re-ranking)	0.857 (18/21)	0.857	0.683

Table 2: Results on the 21 questions. Hit@3 means the gold chunk was one of the 3 chunks given to the LLM.

Re-ranking fixed 6 of the 21 questions and did not break any. 12 questions were fine in both pipelines, and 3 failed in both. Recall@10 is the same for both pipelines because Pipeline 2 only reorders the same 10 chunks. What I found interesting is that Pipeline 2's Hit@3 is equal to Recall@10: every time the gold chunk was somewhere in the top 10, the cross-encoder moved it into the top 3.

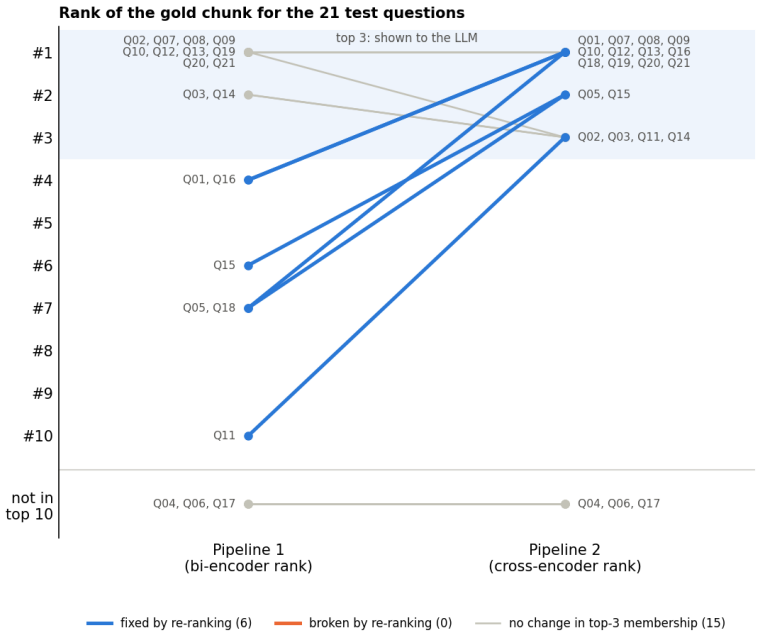


Figure 1: Rank of the gold chunk before (left) and after (right) re-ranking. Blue = fixed by re-ranking.

ID	Question (type)	Rank	Answer 1	Answer 2
Q01	Wife and I work for the same company, can each of us take 12 weeks for our baby? (N)	4 → 1	not in context	correct: spouses share 12 weeks in total
Q16	If I don't come back after leave, do I have to pay back health premiums? (E)	4 → 1	"yes" but based on the wrong rule, no exceptions	correct, including the exceptions
Q11	I'm a flight attendant, how many hours must I have worked? (E, K)	10 → 3	not in context	correct: 504 hours + 60% of monthly guarantee
Q05	Can an employer move staff to keep a site under 50 employees? (N)	7 → 2	not in context	correct: this is prohibited
Q15	Can my employer cancel my health insurance if I stop paying my share? (E)	6 → 2	not in context	correct: yes, if over 30 days late, with notice
Q18	How early must I tell my employer about a planned surgery? (C)	7 → 1	not in context	correct: at least 30 days
Q04	Can my employer count FMLA absences under a no-fault attendance policy? (N)	15 → –	not in context	not in context
Q06	Does my employer always have to give me my job back? (E)	30 → –	not in context	incomplete (misses key-employee exception)
Q17	Can my employer make me use paid vacation during FMLA leave? (C)	21 → –	not in context	not in context

Table 3: Failure queries. Rank = rank of the gold chunk in Pipeline 1 → Pipeline 2 ("–" = not in the top 10). "Not in context" means the model answered "The provided context does not contain the answer."

Q01 (constraint buried at rank 4). The bi-encoder's top 3 were all chunks saying "up to 12 workweeks for the birth of a child", and one of them even says "Fathers are equally entitled to take up to 12 workweeks". The chunk with the actual answer (spouses at the same employer share 12 weeks in total) was rank 4, and its score was only a little lower (0.686 vs 0.720). So the LLM never saw it and Answer 1 was "not in context". The cross-encoder moved this chunk (chunk 140) from #4 to #1, and Answer 2 says correctly that the 12 weeks have to be shared.

Q16 (exception in another chunk). This was the worst baseline answer. Answer 1 did not abstain. It found a nearby "Did you know?" box about the employer recovering the *employee's* share of premiums "whether or not the employee returns", and used that to answer "yes". That is a different rule, and the real rule's exceptions (you don't have to pay if you didn't return because of a serious health condition or something outside your control) were missing. After re-ranking, the right chunk went from #4 to #1 and Answer 2 includes those exceptions. I would call Answer 1 a misstated rule rather than a hallucination, because everything it said is in the guide, just not for this situation.

Q11 (keyword trap). The question talks about hours worked, so the bi-encoder ranked the general "1,250 hours of service" chunks first. The special rule for airline flight crews (504 hours) was only at #10, even though that chunk mentions flight attendants by name. The cross-encoder picked this up and moved it to #3.

Q05 and Q04 (negation). Both answers are "prohibited" bullets inside the Chapter 7 list of prohibitions. That chunk is a long list, so its average vector is about prohibitions in general and the specific bullet gets lost. For Q05 the chunk was still at #7, so re-ranking could rescue it (#7 $\rightarrow$ #2). For Q04 it was at #15, outside the top 10, so re-ranking could not do anything.

Did the baseline hallucinate? Mostly no. Because of the grounded prompt, when the right chunk was missing the model usually said "not in context" (5 of the 6 fixed questions). That is better than making something up, but for the user it is still a failure because the guide does have the answer. The more dangerous cases were Q16 (confident answer from the wrong rule) and Q06 in Pipeline 2, where the answer sounded fine but left out the key-employee exception.

The 3 questions that failed in both pipelines (Q04, Q06, Q17) are recall failures: the gold chunks were at ranks 15, 30 and 21. Two of them are vocabulary problems: the guide says "restoration" and "substitute accrued paid leave", while my questions said "give my job back" and "use my vacation days". One mistake I made myself: the evidence phrase I picked for Q14 ("in loco parentis") was too general and also matched a chunk that does not really answer the question. I kept the original label in the results, but with a stricter phrase Q14 would be a 7th question fixed by re-ranking.

4 Latency profiling

I timed each stage with `time.perf_counter()` on my laptop's CPU. I did a few warm-up runs first, then ran all 21 questions 3 times (63 runs per stage). The Gemini calls were timed separately with real, uncached requests (6 questions $\times$ 2 pipelines).

Stage (ms)	Mean	Median	p95
Query embedding	13.0	13.1	15.8
FAISS search (top 10)	0.1	0.2	0.2
Cross-encoder (10 pairs)	297.4	296.0	328.9
Retrieval, Pipeline 1	13.1	13.3	16.0
Retrieval, Pipeline 2	310.5	310.0	346.4
Gemini answer, Pipeline 1	1120.7	844.0	2173.0
Gemini answer, Pipeline 2	1283.0	1106.0	2335.2
End-to-end, Pipeline 1 (sum of medians)		857	
End-to-end, Pipeline 2 (sum of medians)		1416	

Table 4: Latency per stage. Embedding all 207 chunks is a one-time cost of about 4.6 s and is not included.

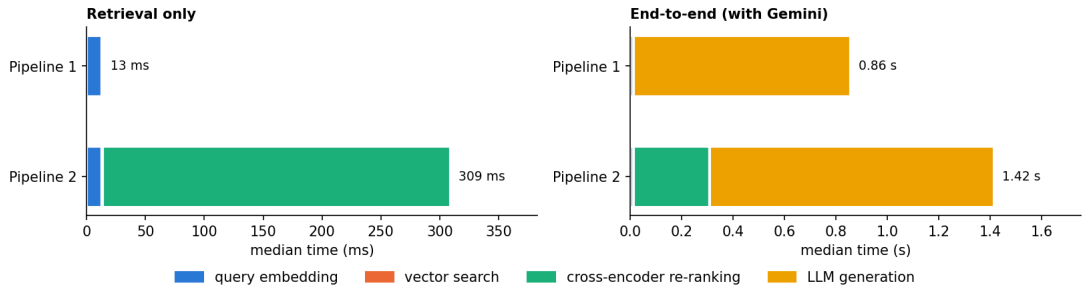


Figure 2: Median time per stage for both pipelines.

Re-ranking makes the retrieval part about 23 times slower (13 ms to 310 ms), and almost all of that time is the cross-encoder doing 10 forward passes. But when the LLM is included, the gap is much smaller (about 0.86 s vs 1.42 s), because the Gemini call takes most of the time anyway. One thing to keep in mind: Pipeline 2's Gemini calls were a bit slower partly because its answers were longer (median 91 tokens vs 9 tokens for the "not in context" replies in Pipeline 1). The Gemini times also vary a lot between calls since they go over the network.

5 Why not use the cross-encoder on the whole collection?

The main reason is that cross-encoder scores cannot be precomputed. The score depends on the question and the chunk together, so for every new question, every chunk needs its own forward pass. With the bi-encoder, the chunks are encoded only once, and a new question only needs one encoding plus cheap vector comparisons.

I measured the cross-encoder at about 29 ms per (question, chunk) pair, and the time grows linearly with the number of pairs. Scoring all 207 chunks of my guide already took 5.7 s per question. If I extrapolate, 10,000 chunks would take about 5 minutes per question and 1 million chunks about 8 hours, while retrieve-10-then-re-rank stays at around 0.3 to 0.4 s. (FAISS exact search took 37 ms for 500,000 vectors in my test; an approximate index like HNSW would be even faster.)

Since my document is small, I could actually try the exhaustive version. It was better: Hit@3 went up to 0.952, because it found Q04 and Q17 at rank 1. But re-ranking just the top 30 instead of the top 10 gave the same Hit@3 (0.952) in about 0.94 s instead of 5.7 s:

Chunks re-ranked	Hit@3	Re-ranking time
top 10	0.857	0.30 s
top 20	0.905	0.62 s
top 30	0.952	0.94 s
all 207	0.952	5.7 s

Table 5: Re-ranking more candidates helps, but the time grows linearly.

So the better approach is to use the bi-encoder to get a candidate list that is big enough for good recall, and the cross-encoder only to order that short list. For a real collection with millions of chunks, running the cross-encoder on everything would just be too slow.

6 Conclusion

The bi-encoder is fast, but it often puts the important chunk just outside the top 3, especially for questions about exceptions, constraints and negations, and then the LLM cannot answer or answers using the wrong rule. Adding the cross-encoder raised Hit@3 from 0.57 to 0.86 on my 21 questions, fixed 6 questions without breaking any, and cost about 0.3 s extra per question on my laptop. Re-ranking cannot fix recall failures, though. For those, re-ranking more candidates (top 30 worked here) or adding keyword search would help.

The main limitations are that I only used one document and 21 questions, the timings are from one laptop, and the gold labels depend on the evidence phrases I chose.